

* Feature selection :-

- minimal setup features that are good enough for a model
↳ we need minimal features to:
(i) reduce computational complexity

• No. of samples $\ll$ No. of features

$$S_1 \quad x_1, x_2, x_3 \dots \dots \dots x_{100,000} \rightarrow y^{(1)}$$

$$S_2 \quad x_1, x_2, x_3 \dots \dots \dots x_{100,000} \rightarrow y^{(2)}$$

$$S_3 \quad x_1, x_2, x_3 \dots \dots \dots x_{100,000} \rightarrow y^{(3)}$$

* If more features, less samples $\rightarrow$ overfitting

* Ideal no. of samples

ML: $10 \times \text{No. of features}$

DL: $10000 \times \text{No. of features}$

* Strategies :-

- $\rightarrow$ LASSO / L1 Regularisation
- $\rightarrow$ Forward Search
- $\rightarrow$ Backward Search
- $\rightarrow$ Recursive Feature Elimination (RFE)

precision and recall

Specificity

ROC

sensitivity

1.0

① Forward search:-

x_1, x_2, x_3, x_4, x_5

Goal is to come down from 'n' no. of features to a smaller number

Step 1:- $F = \{\phi\}$ # empty set F (no. features)

$m_1 = h_0(x) = \theta_0$ # the first model

Step 2:- Add feature x_1 to F

$F = \{\phi, x_1\} \rightarrow m_1 \rightarrow AUC_1$ (scoring matrix)

Repeat this step, pick one feature at a time $\rightarrow$ this tells us how much that

$F = \{\phi, x_2\} \rightarrow m_2 \rightarrow AUC_2$

feature will include the performance of model

$F = \{\phi, x_3\} \rightarrow m_3 \rightarrow AUC_3$

$F = \{\phi, x_4\} \rightarrow m_4 \rightarrow AUC_4$

$F = \{\phi, x_5\} \rightarrow m_5 \rightarrow AUC_5$

Pick feature with maximum AUC

Let us say x_2 performed better - then commit x_2 to feature set
 $F = \{x_2\}$ (final selection list)

Step 3 : $M_1 = \{x_2, x_1\} \rightarrow AUC_1$ # Day x_4 gives better performance
 $M_2 = \{x_2, x_2\} \rightarrow AUC_2$ Then commit it to the
 $M_3 = \{x_2, x_3\} \rightarrow AUC_3$ feature list
 $M_4 = \{x_2, x_4\} \rightarrow AUC_4$ $F = \{x_2, x_4\}$
 $M_5 = \{x_2, x_5\} \rightarrow AUC_5$

Do this until no more performance can be increased or when another feature is added, the performance is decreased. This is a good method but computationally very expensive.

(2) Backward Search : x_1, x_2, x_3, x_4, x_5

- Take all the features at once, then go on by dropping each feature one by one.
- If an important feature is dropped, the sorting metrics change accordingly.
 Start with all the features $F = \{x_1, x_2, x_3, x_4, x_5\}$
 • remove/drop feature one by one.

(3) RFE (similar to backward search) :
 = "feature importance"

- ↳ coefficient values ($\uparrow$ value, $\uparrow$ importance)
- ↳ probability score

instead of metrics we are looking at the score prob. score.

→ Find which feature is most imp, drop it accordingly.
 $F = \{x_1, x_2, \dots, x_5\}$

- Train a model, look at the scores for each feature.
- Remove features with the lowest scores

At each step, we look at scores and eliminate accordingly. $\Rightarrow$ Recurse